# Solution: Kth Smallest Element in a BST (medium)

## Problem Statement

Given a `root` node of the Binary Search Tree (BST) and integer `K`. Return the Kth smallest element among all node values of the binary tree.

**Examples:**

1. **Example 1:**

   **Input:**

   ```
       8
      / \
     3   10
    / \    \
   1   6    14
      / \   /
     4   7 13

   k = 4
   ```

   **Expected Output:** 6

   **Justification:** The in-order traversal of the tree is [1, 3, 4, 6, 7, 8, 10, 13, 14]. The 4th element in this sequence is 6.

2. **Example 2:**

   **Input:**

   ```
       5
      / \
     3   6
    / \
   2

   k = 3
   ```

   **Expected Output:** 5

   **Justification:** The in-order traversal of the tree is [2, 3, 5, 6]. The 2nd element in this sequence is 5.

3. **Example 3:**

   **Input:**

   ```
     1
      \
       3
      /
     2

   k = 2
   ```

   **Expected Output:** 2

   **Justification:** The in-order traversal of the tree is [1, 2, 3]. The 2nd element in this sequence is 2.

## Solution

The most intuitive approach to this problem is to perform an in-order traversal on the BST. Given the nature of BSTs, an in-order traversal will result in a sequence of nodes in ascending order. Thus, by collecting the nodes in a list during traversal, the kth element of this list would be our answer.

However, there is a more efficient approach. We don't necessarily need to traverse the entire BST. We can halt our in-order traversal as soon as we've visited the kth node. This is achieved by using a counter that's incremented every time we visit a node during our traversal. When the counter matches `k`, we've found our kth smallest element.

**Algorithm Walkthrough:**

Given the tree from Example 1:

```
      8
     / \
    3   10
   / \    \
  1   6    14
     / \   /
    4   7 13
```

With `k = 4`, the steps are:

- Start at the root (8). Since there's a left child, move to the left child (3).
- From node 3, again move to its left child (1). It's the leftmost node, so count it as the first smallest element.
- Move up to node 3. It's the second smallest.
- Now move to its right child (6). Since it also has a left child, move to that (4). It's the third smallest.
- Move up to node 6. It's the fourth smallest - which is what we are looking for. Hence, we stop and return 6.

**Here is the visual representation of the algorithm:**



## Code

Here is the code for this algorithm:




## Complexity Analysis

The worst-case time complexity is $O(N)$ where $N$ is the number of nodes, in case we need to visit all nodes. However, on average, we only have to visit $k$ nodes, making it $O(k)$. The space complexity is $O(1)$ if we disregard the space used by the recursion stack, otherwise, it's $O(h)$ where $h$ is the height of the tree.